

I²C- und SPI-Treiber

Vom beschriebenen Busgerät zum sicheren Datentransfer

4.07

Drei Fragen führen vom Bus zum Transfer

01

Wie entsteht das Binding?

Firmwarebeschreibung,
Match-Tabelle und probe()
verbinden Gerät und Treiber.

02

Wie unterscheidet sich der
Transfer?

I²C-Nachrichten und
SPI-Transfers folgen
verschiedenen Busverträgen.

03

In welchem Kontext ist Zugriff
sicher?
Sleep-Fähigkeit, Nebenläufigkeit
und Rückbau bestimmen die
Ausführung.

Lernziel

Am Ende kannst du einen I²C- oder SPI-Treiberpfad lesen, Transfers einordnen und Fehler evidenzbasiert untersuchen.

Vier Übergänge führen zum funktionierenden Busgerät

Beschreiben

Firmwaredaten benennen Gerät, Adresse, Ressourcen und Eigenschaften.

Binden

Bus-Core und Match-Tabelle wählen den passenden Treiber.

Konfigurieren

probe() richtet Protokoll, Ressourcen und Gerätezustand ein.

Übertragen

Bus-API führt Nachrichten aus und liefert überprüfbare Ergebnisse.

BUSPFAD

Identität: Firmware → Device → Match

Lebensdauer: probe() → Betrieb → remove()

Daten: Request → Controller → Gerät → Ergebnis

Adapter, Controller, Device und Driver bilden den Buspfad

KERNSTRUKTUREN

I²C

```
struct i2c_adapter  
struct i2c_client  
struct i2c_driver
```

SPI

```
struct spi_controller  
struct spi_device  
struct spi_driver
```

Rollen im Modell

01 Bus-Master

Adapter oder Controller setzt Transfers auf der Hardware um.

02 Busgerät

Client oder spi_device hält Adresse, Modus und Gerätezustand.

03 Treiber

Match-Tabelle und Callbacks verbinden Protokoll mit dem Gerät.

Firmwarebeschreibung und Match-Tabelle führen zu probe()

BINDING-PFAD

```
Device Tree / ACPI  
compatible / IDs  
modalias  
of_match_table  
probe()  
remove()
```

01

Enumerieren

Firmwaredaten erzeugen ein Busgerät mit Identität und Eigenschaften.

02

Abgleichen

Bus-Core vergleicht IDs, Namen oder compatible-Einträge.

03

Initialisieren

probe() prüft Fähigkeiten, reserviert Ressourcen und startet das Gerät.

Ein sichtbares Device beweist noch kein erfolgreiches Binding – Driver-Link und Log liefern die Evidenz.

I²C und SPI besitzen unterschiedliche Busverträge

BUSSEMANTIK

I²C

adressierte Nachrichten

geteilter Zwei-Draht-Bus

ACK/NACK

SPI

Chip Select

Takt + Datenleitungen

Vollduplex möglich

Zwei Sichtweisen

I²C

Nachrichten adressieren Teilnehmer; kombinierte Transfers halten den Bus zwischen Teiloperationen.

SPI

Der Controller wählt ein Gerät; Mode, Takt und Wortbreite prägen jeden Transfer.

Die Geräte-Spezifikation bestimmt Reihenfolge, Timing und Bedeutung der übertragenen Bytes.

i2c_transfer() führt eine kombinierte Nachrichtenfolge aus

I²C-TRANSFER

```
msgs[0]: Registeradresse schreiben  
msgs[1]: Nutzdaten lesen | I2C_M_RD  
  
ret = i2c_transfer(adap, msgs, 2);  
if (ret != 2)  
    return ret < 0 ? ret : -EIO;
```

Lies als Vertrag

01 Nachrichtenfolge

Write und Read werden als eine zusammenhängende Nachrichtenfolge ausgeführt.

02 Rückgabewert

Erfolg ist die Anzahl vollständig ausgeführter Nachrichten.

03 Geräteprotokoll

Registerbreite, Byte-Reihenfolge und Datenlänge kommen aus dem Datenblatt.

Transferlimits und unterstützte Flags immer gegen Adapter und Kernelversion prüfen.

SMBus-Helfer und Regmap reduzieren Protokollcode

I²C-HILFEN

```
i2c_check_functionality()
i2c_smbus_read_byte_data()
i2c_smbus_write_word_data()
devm_regmap_init_i2c()
regmap_read() / regmap_update_bits()
```

Drei Entscheidungen

01 Fähigkeit

Nur Operationen nutzen, die der Adapter tatsächlich unterstützt.

02 Abstraktion

Regmap kapselt Registerbreite, Cache, Endianness und Zugriff.

03 Fehler

Busfehler unverändert weitergeben und im Device-Kontext melden.

spi_transfer und spi_message beschreiben einen SPI-Zugriff

SPI-TRANSFER

```
struct spi_transfer xfer = {  
    .tx_buf = tx,  
    .rx_buf = rx,  
    .len = len,  
};  
ret = spi_sync_transfer(spi, &xfer, 1);
```

Vom Puffer zum Signal

- 01** Transfer
Puffer, Länge, Geschwindigkeit und Wortbreite beschreiben einen Abschnitt.
Message
- 02** Message
Mehrere Transfers teilen Chip Select und Abschlussstatus.
- 03** Ausführung
spi_sync() blockiert; spi_async() meldet den Abschluss per Callback.

probe() baut Ressourcen in Abhängigkeitsreihenfolge auf

LEBENSDAUER

```
regulator_enable()  
clk_prepare_enable()  
reset_control_deassert()  
devm_regmap_init_*()  
devm_request_threaded_irq()  
pm_runtime_enable()
```

Zwei Ebenen

Initialisierung

Versorgung, Takt, Reset und Registerzugriff geordnet herstellen.

Betrieb

IRQ, Runtime-PM und Gerätezustand über dieselbe Lebensdauer führen.

Managed Resources vereinfachen den Rückbau; aktive Arbeit und Hardwarezustand müssen trotzdem explizit beendet werden.

Buszugriffe benötigen einen sleep-fähigen Kontext

KONTEXT

Hard IRQ

keine Buszugriffe

Threaded IRQ

Transfer möglich

Workqueue

blockierende Arbeit

remove()

Arbeit synchron stoppen

Der sichere Ablauf

01 Auslösen

Hard IRQ quittiert nur das Nötigste und weckt einen Thread.

02 Übertragen

Thread oder Workqueue darf schlafen und serialisiert Gerätezustand.

03 Beenden

Asynchrone Arbeit stoppen, bevor Ressourcen verschwinden.

Mutex, Completion oder zustandsspezifische Sperren müssen zum tatsächlichen Aufrufpfad passen.

Fehler lassen sich nach der erreichten Schicht ordnen

DIAGNOSE

```
kein Device  
kein Driver-Link  
probe()-Fehler  
NACK / -ENXIO  
I/O-Fehler / -EREMOTEIO  
Timeout / -ETIMEDOUT
```

Drei Fehlerklassen

- 01 Binding**
Device, modalias, Match-Tabelle und Driver-Link vergleichen.
- 02 Konfiguration**
Versorgung, Takt, Pinmux, Mode und Busfrequenz prüfen.
- 03 Transfer**
Rückgabewert, Controller-Log und physisches Signal korrelieren.

Praxis: I²C- und SPI-Binding read-only untersuchen

TERMINAL

```
$ readlink -f /sys/bus/i2c/devices/*/driver
$ cat /sys/bus/i2c/devices/*/modalias
$ readlink -f /sys/bus/spi/devices/*/driver
$ dmesg | grep -Ei 'i2c|spi'
```

Belege drei Aussagen

01

Device

Businstanz, Adresse oder Chip Select aus dem Gerätenamen ablesen.

02

Driver

Driver-Link und modalias mit dem Treiber-Match verbinden.

03

Laufzeit

Logmeldungen zeitlich dem Probe- oder Transferpfad zuordnen.

Keine Geräte entbinden und keine Rohtransfers senden: Beobachtung reicht für diese Übung.

Praxis: Einen I²C- oder SPI-Treiberpfad erklären

01**Gerät wählen**

Ein vorhandenes I²C- oder SPI-Gerät und seine Businstanz auswählen.

02**Binding belegen**

Firmwareidentität, modalias, Driver-Link und Match-Tabelle verbinden.

03**Transfer finden**

Die verwendete Bus- oder Regmap-API im Treiber lokalisieren.

04**Kontext prüfen**

Aufrufkontext, Sperren und mögliche Sleep-Punkte markieren.

05**Rückbau erklären**

Asynchrone Arbeit und Ressourcen bis remove() verfolgen.

06**Evidenz teilen**

Pfad, erwartete Fehler und zwei überprüfbare Belege vorstellen.

Read-only arbeiten; keine Busgeräte entbinden oder produktive Hardware verändern.

I²C und SPI verbinden Binding, Protokoll und Lebensdauer.

Formuliere in einem Satz:

Welche Evidenz beweist, dass ein Busgerät korrekt gebunden und sicher angesprochen wird?